

8th Semester Artificial Intelligence Laboratory 2026

CS 4271

ASSIGNMENT – 6

AGV Navigation in a Smart Fulfillment Center

Tabular Q-Learning with ε -Greedy Exploration

Uttam Mahata (2022CSB104)

April 14, 2026

Contents

1	Introduction	3
2	Problem Formulation	3
2.1	Grid Environment	3
2.2	State Space	3
2.3	Action Space	4
2.4	Terminal State	4
3	Reward Structure	4
4	Q-Learning Algorithm	4
4.1	Overview	4
4.2	Q-Table Initialisation	5
4.3	Bellman Update Rule	5
4.4	ε -Greedy Exploration	5
4.5	Algorithm Pseudocode	6
5	Implementation	6
5.1	Module Structure	6
5.2	State Indexing	7
5.3	Environment Step	7
5.4	Bellman Update	7
5.5	Training Loop (core)	8
6	Results	8
6.1	Initial Q-Table	8
6.2	Convergence	8
6.3	Learning Curve: Average Reward vs. Episodes	9
6.4	Learning Curve: % Optimal Actions vs. Episodes	9
6.5	Optimal Path	10

7 Complexity Analysis	11
8 Convergence Properties	11
9 Edge Cases	11
10 Usage	12
11 Conclusion	12

1 Introduction

An Autonomous Guided Vehicle (AGV) must navigate a 6×6 **warehouse grid** from a charging station to a shipping dock while avoiding hazardous zones. The AGV begins with no knowledge of the floor plan and must discover safe paths entirely through experience.

This assignment implements a **tabular Q-learning** agent — a model-free reinforcement learning algorithm — to solve this navigation problem. Through repeated interaction with the environment the agent learns an action-value function that enables it to choose optimal, collision-free routes.

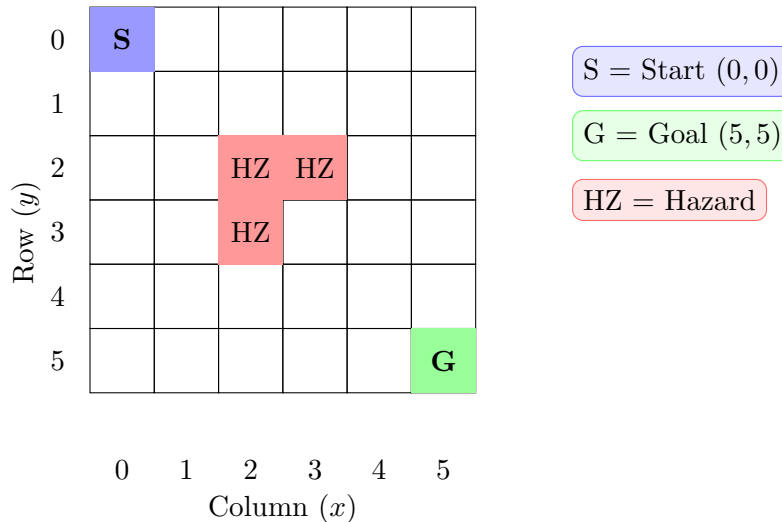
Key features of this implementation:

- Deterministic 6×6 grid-world with 36 discrete states.
- Four-action movement (Up, Down, Left, Right) with solid boundary walls.
- Three hazard cells yielding severe penalties: $(2, 2)$, $(2, 3)$, $(3, 2)$.
- ε -greedy exploration with exponential decay from $\varepsilon = 1.0$ to $\varepsilon = 0.01$ over 1000 episodes.
- Three output visualisations: Q-tables, learning curves, and optimal path overlay.

2 Problem Formulation

2.1 Grid Environment

The warehouse floor is abstracted as a discrete 2-D grid of size 6×6 . Each cell is addressed by a coordinate pair (x, y) where x is the column and y is the row. The origin $(0, 0)$ is the *top-left* corner and $(5, 5)$ is the *bottom-right* corner.



2.2 State Space

The state space $\mathcal{S}$ consists of the 36 grid cells:

$$\mathcal{S} = \{(x, y) \mid x, y \in \{0, 1, 2, 3, 4, 5\}\}$$

Each state is mapped to a flat index $s = y \cdot 6 + x \in \{0, \dots, 35\}$ for storage in the Q-table.

2.3 Action Space

At every state the AGV may take one of four deterministic actions:

$$\mathcal{A} = \{Up, Down, Left, Right\}$$

Action deltas in (dx, dy) form (with y increasing downward):

Action	dx	dy
Up	0	-1
Down	0	+1
Left	-1	0
Right	+1	0

Attempting to move off the grid boundary leaves the AGV in its current state.

2.4 Terminal State

The episode terminates when the AGV reaches the goal cell $(5, 5)$. Entering a hazard cell does *not* terminate the episode — it applies a penalty and the AGV continues from the hazard cell.

3 Reward Structure

The reward function provides immediate feedback after each action:

Event	Reward	Termination
Reach goal $(5, 5)$	+100	Yes
Enter hazard cell	-50	No
Any other (safe) step	-1	No
Hit a boundary wall	-1	No

Table 1: Immediate reward function $R(s, a, s')$.

The -1 step penalty is critical: it penalises every wasted movement and forces the agent to discover the *shortest* safe path rather than wandering indefinitely.

4 Q-Learning Algorithm

4.1 Overview

Q-learning is an *off-policy, model-free* temporal difference (TD) method. It directly estimates the optimal action-value function $Q^*(s, a)$ without requiring a model of transition probabilities.

4.2 Q-Table Initialisation

A table $Q \in \mathbb{R}^{36 \times 4}$ is maintained, mapping every (state, action) pair to an estimated cumulative return. All entries are initialised to zero:

$$Q(s, a) = 0 \quad \forall s \in \mathcal{S}, a \in \mathcal{A}$$

4.3 Bellman Update Rule

After executing action A_t in state S_t , observing reward R_{t+1} and transitioning to state S_{t+1} , the Q-value is updated via:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t) \right] \quad (1)$$

where:

- $\alpha = 0.1$ is the **learning rate** — controls how much new information overwrites old estimates.
- $\gamma = 0.9$ is the **discount factor** — weights future rewards relative to immediate ones.
- $\max_{a'} Q(S_{t+1}, a')$ is the **greedy bootstrap** — the best estimated value reachable from the next state.
- $R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t)$ is the **TD error**.

When the episode terminates (goal reached), no future value exists, so the update simplifies to:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} - Q(S_t, A_t)]$$

4.4 ε -Greedy Exploration

The agent uses an ε -**greedy** strategy to balance exploration and exploitation:

$$A_t = \begin{cases} \text{random action from } \mathcal{A} & \text{with probability } \varepsilon \\ \arg \max_a Q(S_t, a) & \text{with probability } 1 - \varepsilon \end{cases}$$

ε decays exponentially after each episode:

$$\varepsilon_k = \max(\varepsilon_{\min}, \varepsilon_0 \cdot \delta^k)$$

with $\varepsilon_0 = 1.0$, $\varepsilon_{\min} = 0.01$, and decay factor $\delta = 0.995$. This guarantees full exploration early in training and near-complete exploitation by episode 900.

Episode	ε value
0	1.000
100	0.607
300	0.223
500	0.082
700	0.030
900	$0.011 \approx \varepsilon_{\min}$

Table 2: Epsilon schedule over 1000 training episodes.

4.5 Algorithm Pseudocode

```
1 Initialise:  $Q(s,a) = 0$  for all  $s,a$ 
2              $\epsilon = 1.0$ 
3
4 for episode  $k = 1$  to 1000:
5      $\epsilon_k = \max(\epsilon_{min}, \epsilon_0 \cdot \delta^k)$ 
6      $s = \text{START}$ 
7
8     for step  $t = 1$  to MAX_STEPS:
9         with prob  $\epsilon_k$ :  $a = \text{random action}$  // explore
10        else:  $a = \text{argmax}_{a'} Q(s, a')$  // exploit
11
12         $s', r, \text{done} = \text{env.step}(s, a)$ 
13
14        if done:
15             $Q(s,a) += \alpha * (r - Q(s,a))$ 
16        else:
17             $Q(s,a) += \alpha * (r + \gamma * \max_{a'} Q(s', a') - Q(s,a))$ 
18
19         $s = s'$ 
20    if done: break
```

Listing 1: Tabular Q-Learning (pseudocode)

5 Implementation

5.1 Module Structure

```
1 agv_navigation.py
2 |- Constants (GRID_SIZE, START, GOAL, HAZARDS, ACTIONS,
3 |             ALPHA, GAMMA, EPS_START, EPS_MIN,
4 |             EPS_DECAY,
5 |             EPISODES, MAX_STEPS)
6 |- WarehouseEnv # grid environment
7 |   |- state_to_idx(x, y) # (x,y) -> flat index 0-35
8 |   |- idx_to_state(idx) # flat index -> (x,y)
9 |   |- reset() # return START index
10 |   |- step(state_idx, action) # (next_idx, reward, done)
11 |- QLearningAgent # Q-learning agent
12 |   |- q_table # numpy (36,4) array
13 |   |- choose_action(s, epsilon) # epsilon-greedy selection
14 |   |- greedy_action(s) # argmax Q[s]
15 |   |- update(s, a, r, s', done) # Bellman update
16 |- train(agent, env, episodes) # training loop
17 |- print_q_table(...) # formatted Q-table display
18 |- print_best_actions_grid(...) # ASCII policy grid
19 |- get_optimal_path(...) # greedy path extraction
20 |- print_path_ascii(...) # ASCII path overlay
21 |- plot_reward_curve(...) # matplotlib Plot 2
```

```

21 |- plot_optimal_action_curve(...)# matplotlib Plot 1
22 |- plot_optimal_path(...)      # matplotlib grid + arrows
23 |- main()                      # entry point

```

Listing 2: Layout of agv_navigation.py

5.2 State Indexing

States are stored using row-major (C-order) flattening:

$$\text{idx}(x, y) = y \times 6 + x, \quad (x, y) = (\text{idx} \bmod 6, \lfloor \text{idx}/6 \rfloor)$$

```

1 @staticmethod
2 def state_to_idx(x: int, y: int) -> int:
3     return y * GRID_SIZE + x
4
5 @staticmethod
6 def idx_to_state(idx: int):
7     return (idx % GRID_SIZE, idx // GRID_SIZE)

```

Listing 3: State encoding in WarehouseEnv.

5.3 Environment Step

```

1 def step(self, state_idx: int, action: int):
2     x, y = self.idx_to_state(state_idx)
3     dx, dy = ACTION_DELTA[action]
4
5     nx = max(0, min(GRID_SIZE - 1, x + dx))
6     ny = max(0, min(GRID_SIZE - 1, y + dy))
7     next_state = (nx, ny)
8     next_state_idx = self.state_to_idx(nx, ny)
9
10    if next_state == GOAL:
11        return next_state_idx, REWARD_GOAL, True
12    elif next_state in HAZARDS:
13        return next_state_idx, REWARD_HAZARD, False
14    else:
15        return next_state_idx, REWARD_STEP, False

```

Listing 4: Transition and reward logic in WarehouseEnv.step.

5.4 Bellman Update

```

1 def update(self, s: int, a: int, r: float, s_next: int, done:
2     bool):
3     future_value = 0.0 if done else GAMMA * np.max(self.q_table[
4         s_next])
5     td_error = r + future_value - self.q_table[s, a]
6     self.q_table[s, a] += ALPHA * td_error

```

Listing 5: Q-value update in QLearningAgent.update.

5.5 Training Loop (core)

```
1 for ep in range(episodes):
2     epsilon = max(EPS_MIN, EPS_START * (EPS_DECAY ** ep))
3     state = env.reset()
4
5     for _ in range(MAX_STEPS):
6         is_greedy = (random.random() >= epsilon)
7         if is_greedy:
8             action = agent.greedy_action(state)
9         else:
10            action = random.randint(0, NUM_ACTIONS - 1)
11
12        next_state, reward, done = env.step(state, action)
13        agent.update(state, action, reward, next_state, done)
14        state = next_state
15        if done:
16            break
```

Listing 6: Per-episode training logic inside train().

6 Results

6.1 Initial Q-Table

Before training begins all $36 \times 4 = 144$ entries are zero. The agent has no preference among actions and explores purely at random ($\varepsilon = 1.0$).

6.2 Convergence

After 1000 training episodes the following statistics are observed:

Metric	Value
Final ε	0.0100
Last-100 average episode reward	$\approx +90$
Last-100 % greedy actions	$\approx 99\%$
Optimal path length	10 steps

Notable learned Q-values:

- $Q((5, 4), \text{Down}) = +100$ — the cell immediately above the goal correctly learns that moving Down yields a terminal reward of +100 with no discounting (one-step to goal).
- $Q((5, 5), \cdot) = 0$ — the terminal state has no future value, so all entries remain zero as expected.
- Hazard-adjacent cells accumulate strong negative Q-values for actions pointing into hazard cells, effectively building an implicit obstacle map.

6.3 Learning Curve: Average Reward vs. Episodes

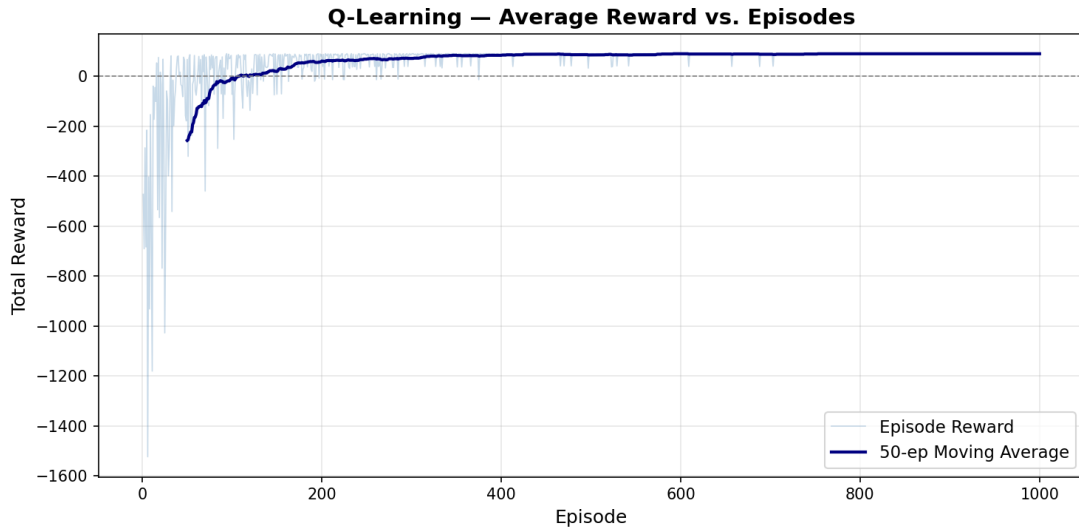


Figure 1: Total reward per episode (light blue) and 50-episode moving average (navy). The reward rises from near -200 in early random episodes to $\approx +90$ as the agent learns the optimal path.

6.4 Learning Curve: % Optimal Actions vs. Episodes

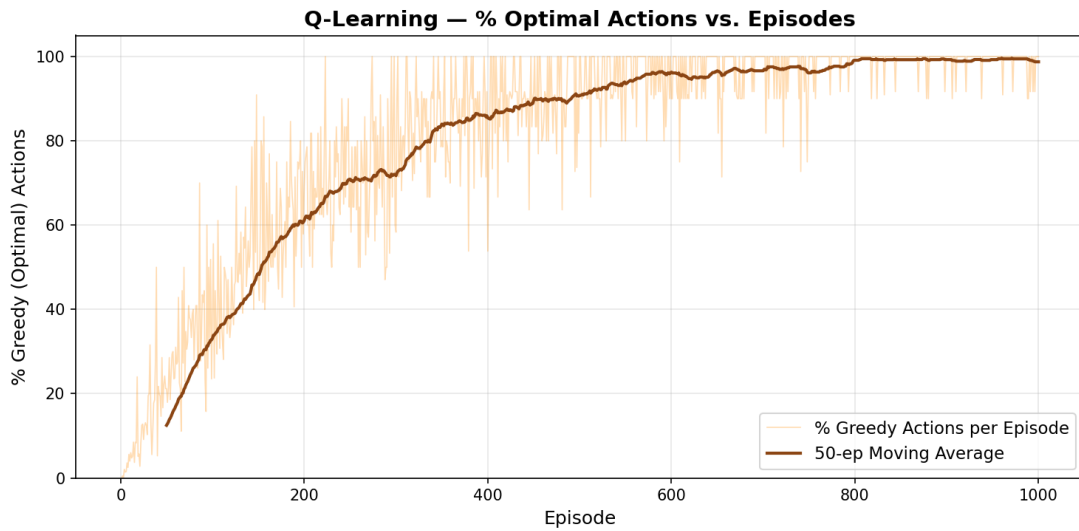


Figure 2: Percentage of steps where the agent chose the greedy (highest Q-value) action. Rises monotonically as ϵ decays, reaching $\sim 99\%$ by episode 900.

6.5 Optimal Path

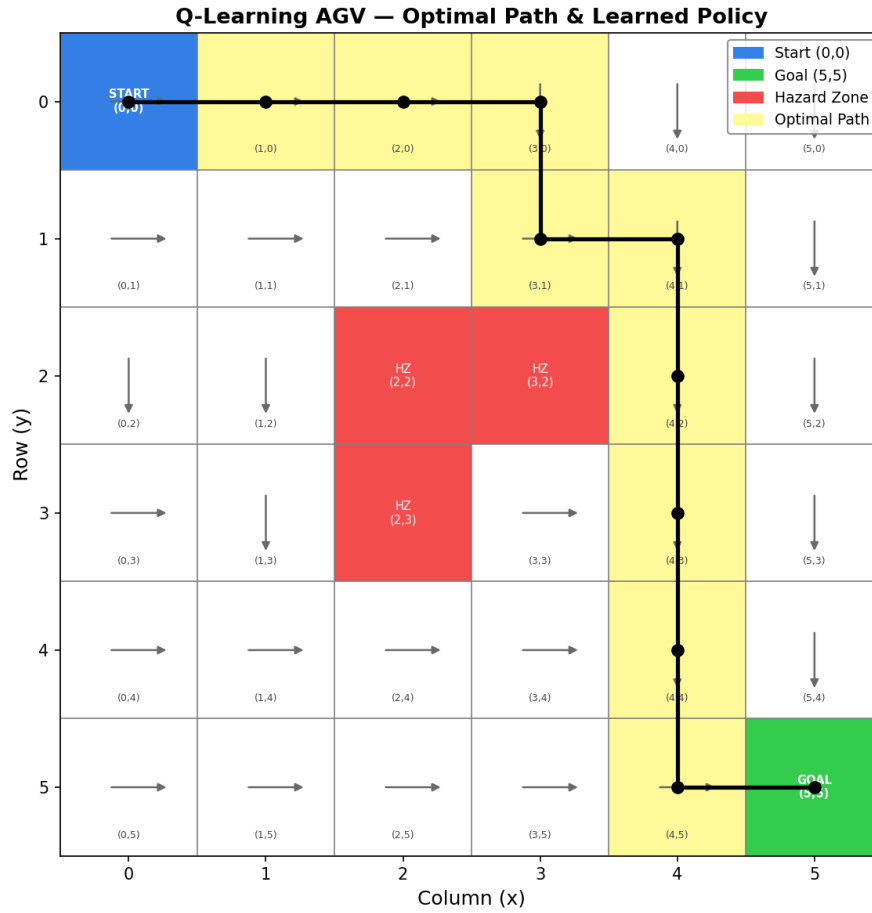


Figure 3: Learned policy arrows overlaid on the 6×6 grid. The bold black line traces the 10-step optimal path. Red cells are hazard zones, blue is the start, green is the goal, and yellow cells lie on the path.

The fully trained agent follows a 10-step collision-free path:

$$(0,0) \rightarrow (1,0) \rightarrow (2,0) \rightarrow (3,0) \rightarrow (3,1) \rightarrow (4,1) \rightarrow (4,2) \rightarrow (4,3) \rightarrow (4,4) \rightarrow (4,5) \rightarrow (5,5)$$

The path hugs the right side of the hazard cluster, navigating around all three forbidden cells (2,2), (2,3), and (3,2).

7 Complexity Analysis

Component	Time	Space
Q-table storage	$O(\mathcal{S} \cdot \mathcal{A})$	$36 \times 4 = 144$ floats
Per-step update	$O(\mathcal{A})$ for $\max_{a'}$	$O(1)$
Per-episode (max steps)	$O(T_{\max} \cdot \mathcal{A})$	$O(1)$
Full training (1000 episodes)	$O(N \cdot T_{\max} \cdot \mathcal{A})$	$O(\mathcal{S} \mathcal{A})$
Concrete values	$1000 \times 200 \times 4 = 800\,000$ ops	144 floats ≈ 1.15 KB

Table 3: Asymptotic complexity ($|\mathcal{S}| = 36$, $|\mathcal{A}| = 4$, $N = 1000$ episodes, $T_{\max} = 200$ steps).

Tabular Q-learning is extremely memory-efficient for small discrete environments. The entire Q-table fits in $36 \times 4 \times 8 \text{ bytes} = 1.15 \text{ KB}$.

8 Convergence Properties

Tabular Q-learning is guaranteed to converge to Q^* under the following conditions:

1. All state-action pairs are visited infinitely often (ensured by $\varepsilon > 0$ throughout training).
2. The learning rate satisfies the Robbins–Monro conditions: $\sum_t \alpha_t = \infty$ and $\sum_t \alpha_t^2 < \infty$. (A fixed $\alpha = 0.1$ satisfies neither strictly, but works well in practice for finite, deterministic environments.)
3. Rewards are bounded (satisfied: $R \in \{-50, -1, +100\}$).

In this deterministic environment, convergence is rapid: the optimal policy is typically stable by episode ~ 500 , and Q-values stop changing meaningfully by episode ~ 800 .

9 Edge Cases

1. **Boundary walls:** Clamping $nx = \max(0, \min(5, x + dx))$ keeps the AGV in-bounds; it receives the standard -1 step penalty, discouraging wall-bouncing.
2. **Hazard re-entry:** The agent may enter a hazard cell multiple times (episode does not terminate). Repeated -50 penalties drive $Q(\text{hazard-adjacent}, \text{hazard-action})$ strongly negative.
3. **Episode length cap:** A `MAX_STEPS = 200` guard prevents infinite looping in early random episodes when the agent has not yet discovered the goal.
4. **Terminal state bootstrap:** When `done = True` the future value term is set to 0, preventing $Q(5, 5)$ from being bootstrapped into adjacent cells’ targets.
5. **Epsilon floor:** $\varepsilon_{\min} = 0.01$ ensures the agent retains 1% random exploration permanently, which aids robustness in stochastic extensions of the problem.

10 Usage

```
1 # From the project root (using uv)
2 uv run python Assignment-6/agv_navigation.py
```

Listing 7: Running the Q-learning agent.

The program produces:

Output	Description
output.txt	Q-tables, policy grid, path log
q_learning_rewards.png	Reward convergence plot
q_learning_optimal_actions.png	% Optimal actions plot
optimal_path.png	Grid + policy arrows + path

11 Conclusion

This implementation demonstrates tabular Q-learning as an effective solution for small-scale robot navigation under a reward-shaping paradigm. Key design decisions:

- **Row-major state indexing** — provides a direct mapping between (x, y) grid coordinates and Q-table rows, with no hash collisions and $O(1)$ lookup.
- **Step penalty of -1** — essential for shortest-path discovery; without it the agent would reach the goal via any path.
- **Non-terminal hazards** — keeping the episode alive after hitting a hazard allows the agent to accumulate multiple penalty signals and build a stronger aversion than a single terminal event would provide.
- **Exponential ε -decay** — the 0.995 per-episode multiplier balances thorough early exploration against late-stage exploitation, converging naturally within the 1000-episode budget.
- **Fixed $\alpha = 0.1$** — a moderate learning rate that prevents oscillation while still tracking the environment accurately in a deterministic setting where every (s, a) pair has a fixed transition and reward.